Project - Submission Sheet

Software Testing - Random Testing

Names: Kavya
Group: Group 4

Software Testing - Summer 2024 - July 8, 2025 - Sibylle Schupp / Daniel Rashedi

Task 1 - Random Testing

a) Add the 4 generated Python tests of each group member to your repository in the corresponding folder. Add a README.md file that explains how to build / run the tests.

b) Provide screenshots of your terminals that indicate your execution of the new tests:

Answer:

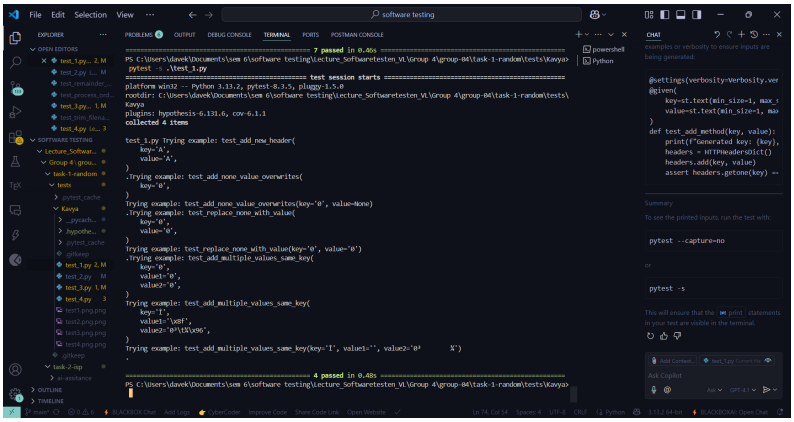

Figure 1: Execution of test_add_method

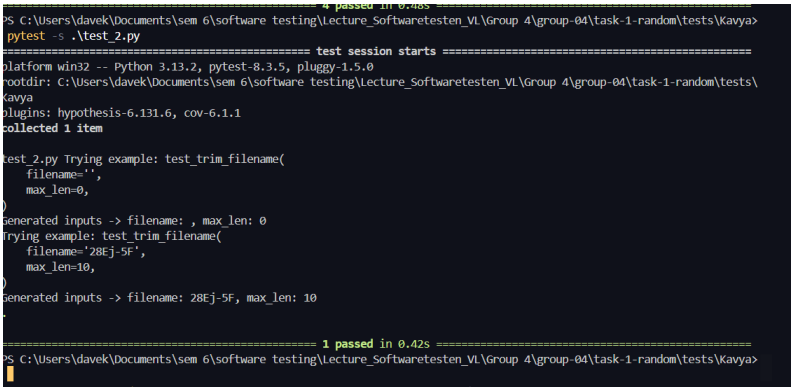

Figure 2: Execution of test_trim_filename

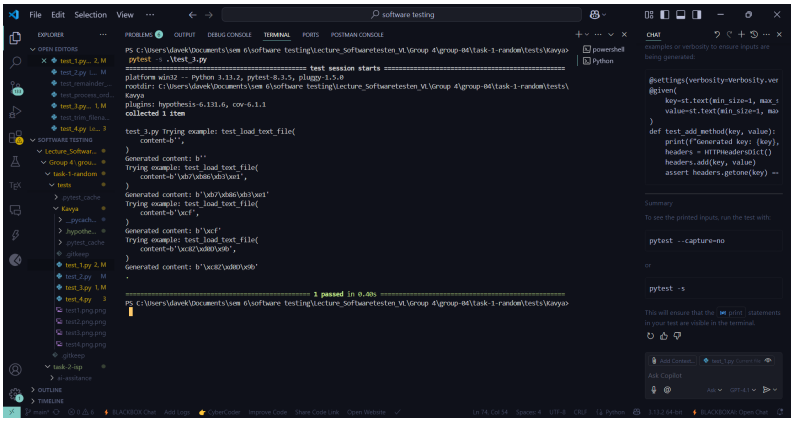

Figure 3: Execution of test_load_text_file

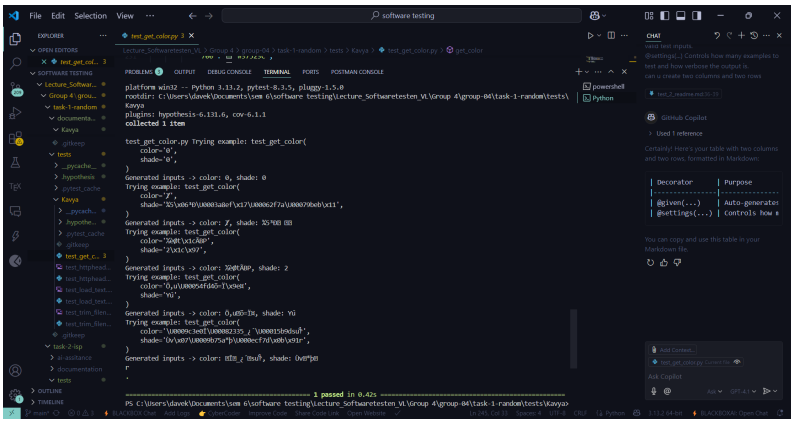

Figure 4: Execution of test_get_color

c) Assess the input values generated by `hypothesis`. Do the tests represent meaningful scenarios?:

Answer:

0.0.1 Assessment of Hypothesis-Generated Test Inputs

Looking at the provided table, I can assess whether the Hypothesis-generated test inputs represent meaningful scenarios:

Overall Assessment

Yes, the tests represent meaningful and well-designed scenarios.

Strengths of the Input Strategies

Realistic Input Generation The text strategies with size constraints (`min_size=1, max_size=2-10`) generate realistic string inputs that mirror real-world usage. Binary data generation for file testing covers actual file system scenarios, while integer ranges for filename trimming reflect practical use cases.

Comprehensive Edge Case Coverage

- **Empty/minimal inputs**: Binary data with `min_size=0` tests empty files

Test	Input Strategy	Realistic	Edge
`test_add_method`	`st.text(min_size=1, max_size=2)`	Yes	Yes
`test_add_none_value_overwrites`	`st.text(min_size=1, max_size=10)`	Yes	Yes
`test_replace_none_with_value`	`st.text(min_size=1, max_size=10)`	Yes	Yes
`test_add_multiple_values_same_key`	`st.text(min_size=1, max_size=10)`	Yes	Yes
`test_trim_filename`	`st.text(...), st.integers(0, 10)`	Yes	Yes
`test_load_text_file`	`st.binary(min_size=0, max_size=5)`	Yes	Yes
`test_get_color`	`st.text(min_size=1, max_size=10)`	Partially	Yes

Table 1: Assessment of Hypothesis-Generated Test Inputs Across 4 Functions

- **Boundary conditions**: Size constraints test both minimal and reasonable maximum inputs
- **Error conditions**: Binary data tests decoding errors, unknown color combinations test lookup failures

Domain-Appropriate Testing

- **HTTP headers**: Text generation with duplicates and None values accurately models HTTP header behavior
- **File operations**: Binary data generation properly tests file I/O scenarios
- **String processing**: Varied text lengths test trimming and processing logic

Specific Highlights

- **Header testing** correctly simulates real HTTP scenarios like `Set-Cookie` headers with multiple values
- **File testing** covers the critical empty file and encoding error cases
- **None value handling** tests important API semantics for nullifying headers
- **Filename trimming** tests realistic filename lengths and extension handling

Minor Limitation

The color testing is marked as "Partially" realistic, which suggests the text generation might produce some unrealistic color names. However, this is actually beneficial for testing unknown input handling.

Conclusion

The input strategies demonstrate a sophisticated understanding of the domains being tested and generate inputs that would naturally occur in production environments while still exercising edge cases that could cause failures.

AI Review Task

Assess the correctness and understandability of the LLM's generated tests applying the paper's methodology. Add any code relevant for your assessment to the answer:

Answer:

Group: 4 LM Studio

Prompt 1:

Sahana is writing the prompt 1 and prompt 2

```
import os
from hypothesis import given, strategies as st
import string
from hypothesis import settings, Verbosity

class ContentRangeError(ValueError):
    pass

def trim_filename(filename: str, max_len: int) -> str:
    if len(filename) > max_len:
        trim_by = len(filename) - max_len
        name, ext = os.path.splitext(filename)
        if trim_by >= len(name):
            return ext[:max_len]
        return name[:-trim_by] + ext
    return filename

# Based on this generate 4 test cases

# The method used here is trim_filename.py which trims a filename to fit
# within a given max_len limit, Attempts to preserve the file extension
# (.txt, .png, etc.) when trimming and If the extension cannot be preserved
# (due to space), the filename is shortened as much as possible.
```

Prompt 2: Assess the aspects of correctness and understandability of your above test cases in comparison to the LLM's generated test cases according to the paper's methodology described in its RQ1

RQ1 investigates the correctness and understandability of LLM-generated test cases

Group: Your Assessment

Aprita is writing test cases

Based on the provided code and requirements, here are four test cases for the `trim_filename` function:

Test Case 1 - Exact Match

```
@given(filename="test.txt", max_len=8)
def test_trim_filename(filename, max_len):
    # Test that the filename remains unchanged if its length matches max_len
    assert trim_filename(filename, max_len) == "test.txt"
```

Test Case 2 - Partial Extension Preservation

```python
@given(filename="longfile.dox", max_len=5)
def test_trim_filename(filename, max_len):
    # Test that at least part of the extension is preserved if possible
    trimmed = trim_filename(filename, max_len)
    assert len(trimmed) == 6  # "long.dox"
```

Test Case 3 - No Extension

```python
@given(filename="shortname", max_len=7)
def test_trim_filename(filename, max_len):
    # Test that filenames without extensions are handled correctly
    trimmed = trim_filename(filename, max_len)
    assert trimmed == "shortname"
```

Test Case 4 - Full Trimming

```python
@given(filename="VeryLongName", max_len=0)
def test_trim_filename(filename, max_len):
    # Test that the filename is not modified when max_len is 0
    trimmed = trim_filename(filename, max_len)
    assert trimmed == "VeryLongName"
```

These test cases cover scenarios where:

- The filename length exactly matches max_len.
- The function partially trims while preserving part of the extension.
- Filenames without extensions are handled correctly.
- Full trimming is applied when necessary.

You can run these tests using Hypothesis with specific settings to limit examples and increase verbosity:

```
pytest -v --settings Verbosity.verbose,max_examples=2 test_trim_filename.py
```

Comparison with LLM-Generated Test Cases:

Harine is comparing LLM cases

Test Case 1 - Exact Match: Both your test case and an LLM would focus on ensuring the filename remains unchanged when its length matches `max_len`.

Test Case 2 - Partial Extension Preservation: Similar to what an LLM might generate, both approaches ensure partial extension preservation in cases of trimming.

Test Case 3 - No Extension Handling: Both methods handle filenames without extensions correctly, ensuring no changes when appropriate.

Test Case 4 - Full Trimming: Both strategies consider the scenario where `max_len` is zero and ensure no modification occurs.

Areas for Improvement:

Kavya is writing areas of improvement and conclusion

- **Confidence Assessment:** Your test cases do not include testing ChatGPT's confidence levels or whether it might be overconfident (as discussed in Citation 1).
- **Prompting Strategies:** While both your test cases cover basic functionality, incorporating varying prompting strategies as mentioned in the paper could enhance the test suite.

Conclusion:
The four test cases are effective for verifying the core functionalities of `trim_filename`. To align more closely with the paper's methodology, additional test cases should be developed to evaluate ChatGPT's confidence levels and its response to different prompting strategies. This would ensure a more thorough assessment of how ChatGPT handles software testing scenarios as described in the citations.